

04 Tests 測試文件

測試目的

本專案是 ICU 護理分配決策支援系統，前端使用 React、TypeScript、Vite，後端使用 Node.js HTTP server 與 PostgreSQL API。測試重點放在三個方向：

- 1. 以單元測試驗證分床與麻煩度計算邏輯，避免核心決策資料算錯。
- 2. 以 API 整合測試確認後端 server 可啟動，且 health endpoint 能回傳標準 JSON 格式。
- 3. 以靜態檢查與建置測試確認程式碼品質、型別與 production build 都可正常通過。
- 4. 以 UI 流程驗收案例確認主要使用者操作路徑可被完整檢查。

測試環境

項目	內容
專案分支	backend-postgres-api
Frontend	React + TypeScript + Vite
Backend	Node.js HTTP server
API base path	/api/v1
測試日期	2026-06-07
測試工具	Vitest、ESLint、TypeScript、Vite、curl

已執行測試總覽

測試類型	測試項目	指令或方法	結果
單元測試	分床計算、麻煩度分數、顯示摘要	npm run test:unit	通過，4 files / 12 tests
API 整合測試	Backend health endpoint	npm run test:api	通過，1 file / 1 test
自動化測試總執行	單元測試 + API 整合測試	npm test	通過，5 files / 13 tests
靜態程式碼測試	ESLint 檢查	npm run lint	通過
建置測試	TypeScript + Vite production build	npm run build	通過

1. 單元測試

測試目標

單元測試針對系統中最容易影響決策正確性的純邏輯函式。這些函式不依賴畫面操作或資料庫，因此適合用自動化測試快速驗證。

測試檔案

測試檔案	測試內容
<code>src/components/allocation/allocationUtils.test.ts</code>	病床顯示格式、護理師負荷風險等級、負荷差值格式
<code>src/components/allocation/allocationBoardUtils.test.ts</code>	各護理師負荷總分、床數、平均負荷、最大最小差距、是否偏離建議分配
<code>src/lib/burdenFactors.test.ts</code>	客觀麻煩度總分、主觀麻煩度總分、未填欄位檢查
<code>src/lib/burdenDisplay.test.ts</code>	麻煩度明細文字、摘要文字、分數高低風險分類

測試程式碼副本

為了方便報告審查，已將本次撰寫的測試程式碼同步放在 `docs/04_tests/test-code/`。正式執行測試時仍以專案原始位置為準，也就是 `src/**/*.test.ts` 與 `tests/api/**/*.test.ts`。

執行方式

```
npm run test:unit
```

通過標準

- 分床負荷總分必須等於病人分數加總。
- 平均負荷、最大值、最小值、差距必須符合預期。
- 手動調整分床時，必須能正確判斷是否偏離系統建議。
- RASS、管路、下床風險、換藥與監測頻率必須正確換算成主觀麻煩度分數。
- 分數高低風險必須符合 UI 顯示門檻。

執行結果

本次執行結果為通過：

```
Test Files  4 passed (4)
Tests       12 passed (12)
```

2. API 整合測試

測試目標

確認後端 API server 可以正常啟動，並且 `/api/v1/health` endpoint 可以回傳標準 JSON response。這是最基本的後端整合測試，用來確認：

- Node.js API server 可以成功 listen。

- API route 可以正確被 dispatch。
- JSON response 格式符合專案規格。
- 前端開發時可透過 Vite proxy 呼叫 `/api/v1`。

測試檔案

測試檔案	測試內容
<code>tests/api/health.test.ts</code>	測試時啟動臨時 API server，呼叫 <code>/api/v1/health</code> ，確認 HTTP status 與 JSON body

執行方式

```
npm run test:api
```

此測試會使用臨時 port 啟動後端 server，測完後自動關閉，不會依賴手動開 server。

預期結果

```
{
  "data": {
    "ok": true
  }
}
```

執行結果

本次執行結果為通過：

```
Test Files  1 passed (1)
Tests       1 passed (1)
```

實際驗證內容包含 HTTP `200` 與下列 JSON：

```
{
  "data": {
    "ok": true
  }
}
```

3. 自動化測試總執行

執行方式

```
npm test
```

執行結果

本次執行結果為通過：

```
Test Files  5 passed (5)
Tests       13 passed (13)
```

4. 靜態程式碼測試

測試目標

確認前端程式碼符合 ESLint 規則，避免以下問題：

- React hooks 使用錯誤。
- TypeScript / TSX 基本語法錯誤。
- 未使用變數、錯誤 import、或其他容易造成維護問題的寫法。
- React Refresh 相關限制違反。

執行方式

```
npm run lint
```

通過標準

ESLint 執行完成，沒有 error 輸出，exit code 為 0。

執行結果

本次執行結果為通過。這代表目前程式碼在靜態規則檢查下沒有阻擋展示或建置的錯誤。

5. 建置測試

測試目標

確認整個前端專案可以被 TypeScript 與 Vite 正常編譯成 production 版本。此測試可驗證：

- TypeScript 型別可以通過 `tsc -b`。
- React 頁面與元件 import 關係正確。
- Vite 可以正確處理 CSS、assets、routing 入口與 bundle。
- 實際展示版本不會因編譯錯誤無法產生。

執行方式

```
npm run build
```

此指令等同於：

```
tsc -b && vite build
```

通過標準

- TypeScript 編譯成功。
- Vite build 成功。
- **dist/** 可以產生 production 靜態檔案。

執行結果

本次執行結果為通過，build 輸出包含：

檔案	說明
dist/index.html	前端入口 HTML
dist/assets/*.css	打包後樣式
dist/assets/*.js	打包後 JavaScript

這表示目前專案具備可展示、可部署的 production build。

6. UI 流程驗收測試

除了指令型測試，本專案也有針對主要使用者流程設計手動 UI 驗收測試。由於此系統包含護理師端與小組長端操作，UI 測試重點放在「資料是否正確呈現」與「操作流程是否能完成」。

UI 測試案例

編號	頁面 / 流程	測試步驟	預期結果
UI-01	護理師總覽 /nurse/overview	進入首頁，確認系統自動導向護理師總覽	可看到目前班別、病人與待處理資訊
UI-02	STAT 任務 /nurse/stat	進入 STAT 任務頁，檢查待處理任務清單	任務依病床與狀態呈現，完成狀態可被辨識
UI-03	麻煩度填寫 /nurse/burden-form	修改病人的主觀照護項目並送出	表單狀態更新，分數與狀態不會出現錯誤
UI-04	小組長分床 /leader/allocation	產生分床建議，拖曳病人到不同護理師欄位	各護理師負荷分數即時更新，未分配清單同步變化
UI-05	分床確認 /leader/allocation-result	確認分床結果後進入結果頁	可看到各護理師分配病床與總負荷
UI-06	戰情室 /leader/war-room	進入戰情室檢查護理師負荷與任務狀態	可看到 pending / done / urgent 等整體狀態
UI-07	交班快照 /leader/handover-snapshots	開啟歷史交班快照	可看到快照列表與交班摘要資料

UI 測試通過標準

- 頁面可以正常載入，沒有白畫面。
- 主要資料區塊可以顯示，不出現 undefined、NaN 或錯誤訊息。
- 按鈕、表單、拖曳與確認流程可以依設計操作。
- API 錯誤時有清楚錯誤訊息，不會造成整個頁面崩潰。
- 分床相關頁面在資料變更後，負荷分數與未分配病人數會同步更新。

7. 測試覆蓋的高風險功能

本次測試特別涵蓋下列高風險區域：

功能	風險	對應測試
分床負荷計算	分數錯誤會影響小組長決策	<code>allocationBoardUtils.test.ts</code> 、UI-04
麻煩度分數計算	分數錯誤會影響分床與照護優先順序	<code>burdenFactors.test.ts</code> 、 <code>burdenDisplay.test.ts</code> 、UI-03
API response 格式	前後端 contract 不一致會導致頁面錯誤	<code>tests/api/health.test.ts</code> 、建置測試
React routing	錯誤路由會造成展示時找不到頁面	建置測試、UI-01
表單資料更新	麻煩度填寫錯誤會影響後續分床	UI-03
戰情室與交班資料	跨頁資料呈現錯誤會影響交班判斷	UI-06、UI-07

8. 測試結論

本專案已完成自動化單元測試、API 整合測試、靜態程式碼檢查與 production build 測試。測試範圍涵蓋分床負荷計算、麻煩度計算、API 基本可用性、React / TypeScript 編譯，以及主要 UI 驗收流程。

目前所有自動化測試與品質檢查皆通過，可作為期末展示與報告驗收的測試證明。